

# Machine Learning Classification Algorithms

Dr. Oybek Eraliev,

---

Department of Computer Engineering

Inha University In Tashkent.

Email: [oybekeraliev7@gmail.com](mailto:oybekeraliev7@gmail.com)



# Recap (Task for extra points)

- Apply Linear Regression (Gradient Descent Approach).
- $Lr = 0.0001$ ,  $Num\_iter = 1000$ .
- Print (theta parameters)

| $x_1$ | $x_2$ | $x_3$ | $y$      |
|-------|-------|-------|----------|
| 9     | 12    | 5     | $34 + k$ |
| 13    | 11    | 3     | $39 + k$ |
| 19    | 18    | 3     | $42 + k$ |
| 22    | 23    | 2     | $55 + k$ |
| 25    | 5     | 6     | $68 + k$ |

# Content

---

- **Logistic Regression**
- **Multi-class classification**
- **Regularization**
- **Implementation of Logistic Regression**

# Classification

## Examples:

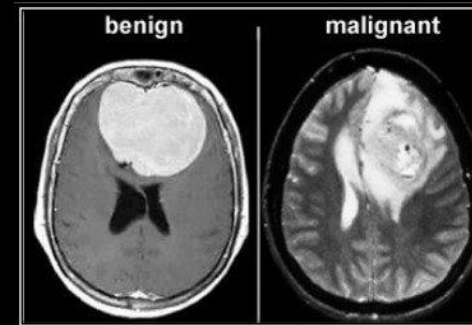
Email: Spam / Not Spam



Online transactions: Fraud Detection (Yes / No)



Brain tumor detection: Benign / Malignant



# Classification

---

## Examples:

Email: Spam / Non-Spam

Online transections: Fraud Detection (Yes / No)

Brain tumor detection: Benign / Malignant

$$y \in \{0, 1\}$$

Binary class

0: “Negative Class” (e.g., benign tumor)

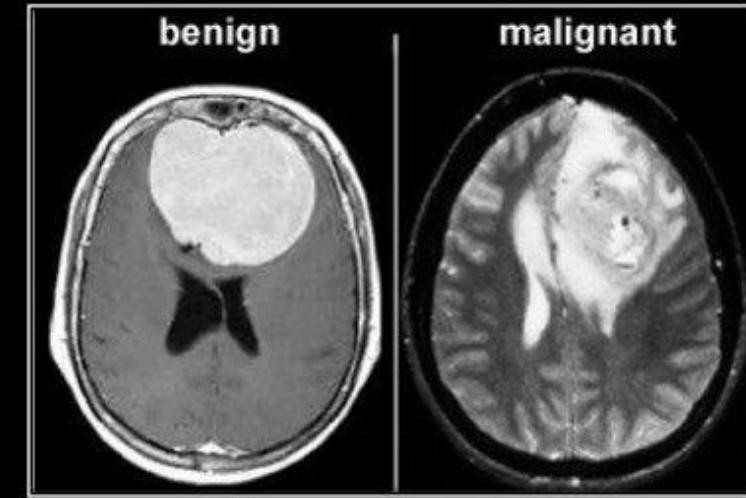
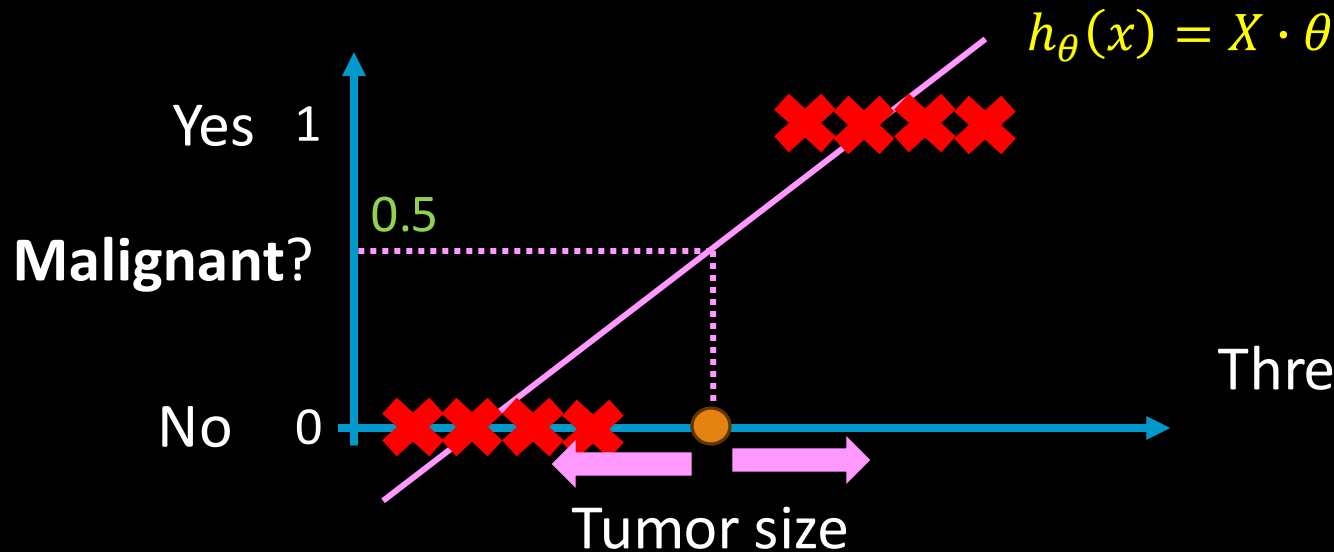
1: “Positive Class” (e.g., malignant tumor)

$$y \in \{0, 1, 2, 3\}$$

Multi-class

# Classification

Examples: Brain tumor detection: Benign / Malignant



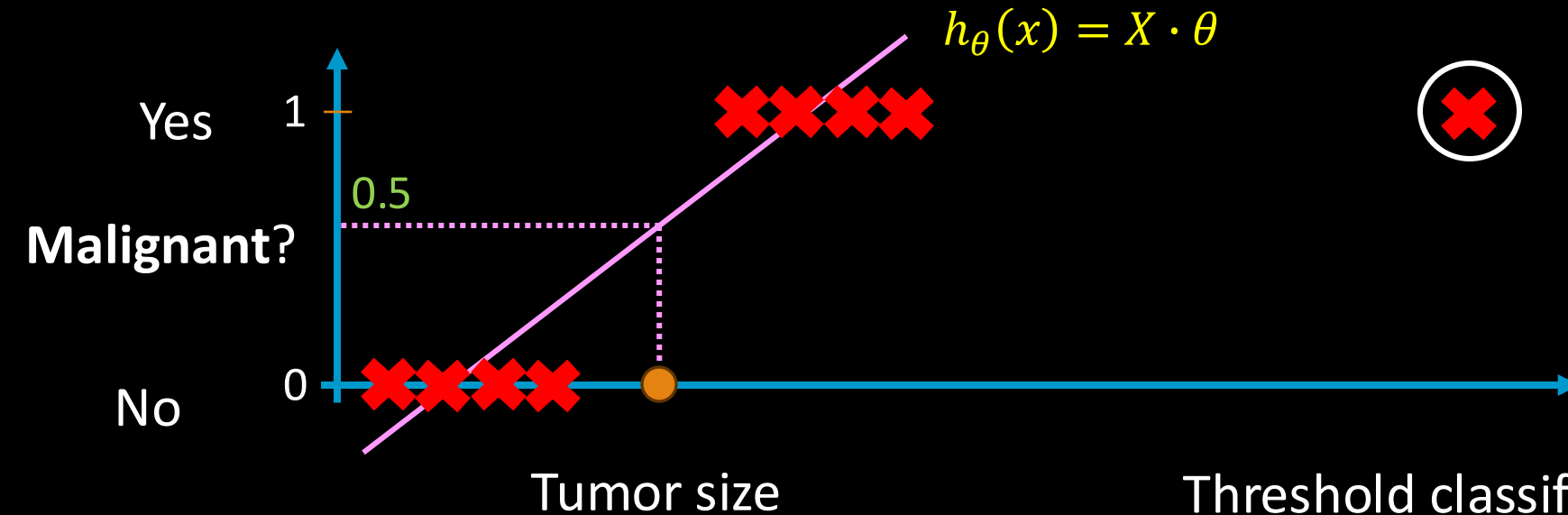
Threshold classifier output  $h_{\theta}$  at 0.5:

If  $h_{\theta}(x) \geq 0.5$ , predict “ $y = 1$ ”

If  $h_{\theta}(x) < 0.5$ , predict “ $y = 0$ ”

# Classification

Examples: Brain tumor detection: Benign / Malignant



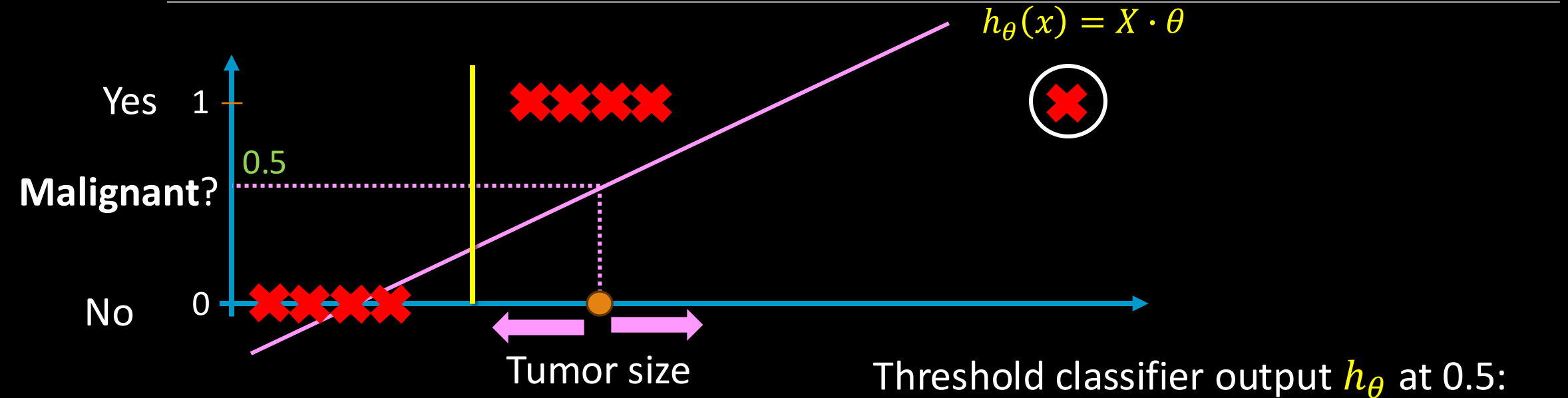
Threshold classifier output  $h_{\theta}$  at 0.5:

If  $h_{\theta}(x) \geq 0.5$ , predict “ $y = 1$ ”

If  $h_{\theta}(x) < 0.5$ , predict “ $y = 0$ ”

# Classification

Examples: Brain tumor detection: Benign / Malignant



If  $h_{\theta}(x) \geq 0.5$ , predict “ $y = 1$ ”

If  $h_{\theta}(x) < 0.5$ , predict “ $y = 0$ ”



# Classification

---

Classification output :  $y = 1$  or  $0$

$$h_{\theta}(x) > 1 \text{ or } h_{\theta}(x) < 0$$

Logistic Regression:  $0 \leq h_{\theta}(x) \leq 1$

# Hypothesis Representation

Logistic Regression model:

Want

$$0 \leq h_{\theta}(x) \leq 1$$

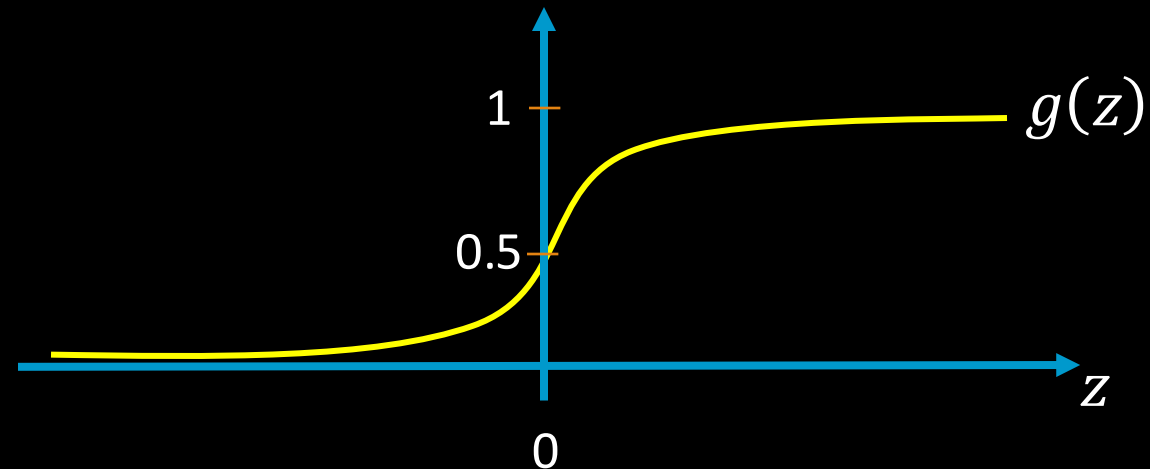
$$h_{\theta}(x) = \frac{1}{1 + e^{-x \cdot \theta}}$$

$$h_{\theta}(x) = g(X \cdot \theta)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

**Logistic Function**

**Sigmoid function**



# Interpretation of Hypothesis Output

$h_{\theta}(x)$  estimated probability that  $y = 1$  n input  $x$

---

**Example:** If  $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ tumorSize \end{bmatrix}$

$$h_{\theta}(x) = 0.7 \qquad y = 1$$

Tell patient that 70% chance of tumor being malignant

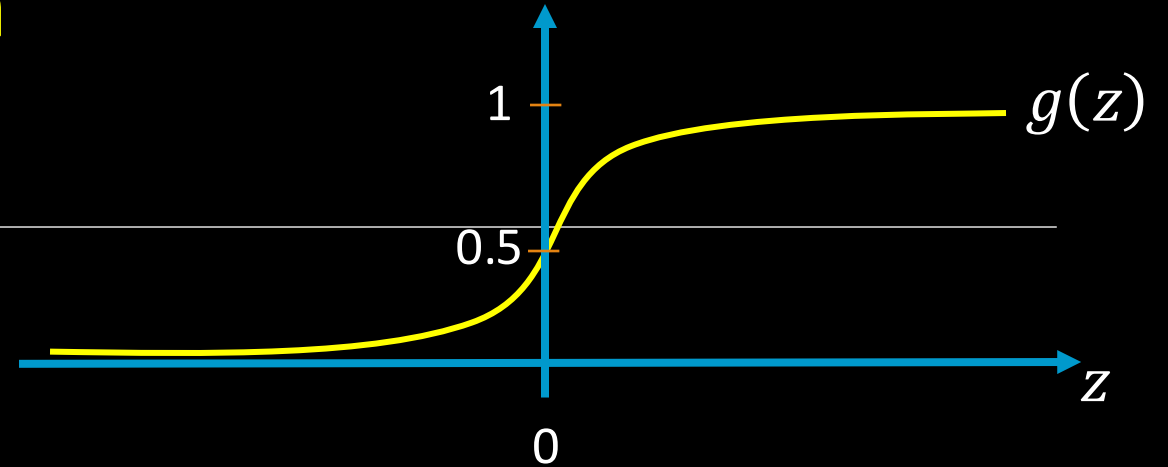
$$h_{\theta}(x) = P(y = 1|X; \theta) \qquad P(y = 0|X; \theta) + P(y = 1|X; \theta) = 1$$

$$y = 0 \text{ or } 1 \qquad P(y = 0|X; \theta) = 1 - P(y = 1|X; \theta)$$

# Hypothesis Representation

$$h_{\theta}(x) = g(X \cdot \theta)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$



Threshold classifier output  $h_{\theta}$  at 0.5:

If  $h_{\theta}(x) \geq 0.5$ , predict “ $y = 1$ ”

If  $h_{\theta}(x) < 0.5$ , predict “ $y = 0$ ”

$$g(z) \geq 0.5 \text{ if } z \geq 0$$

$$g(z) < 0.5 \text{ if } z < 0$$

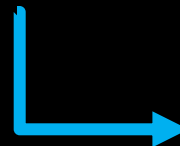
# Decision Boundary

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$\downarrow \quad \downarrow \quad \downarrow$   
 $-3 \quad 1 \quad 1$

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

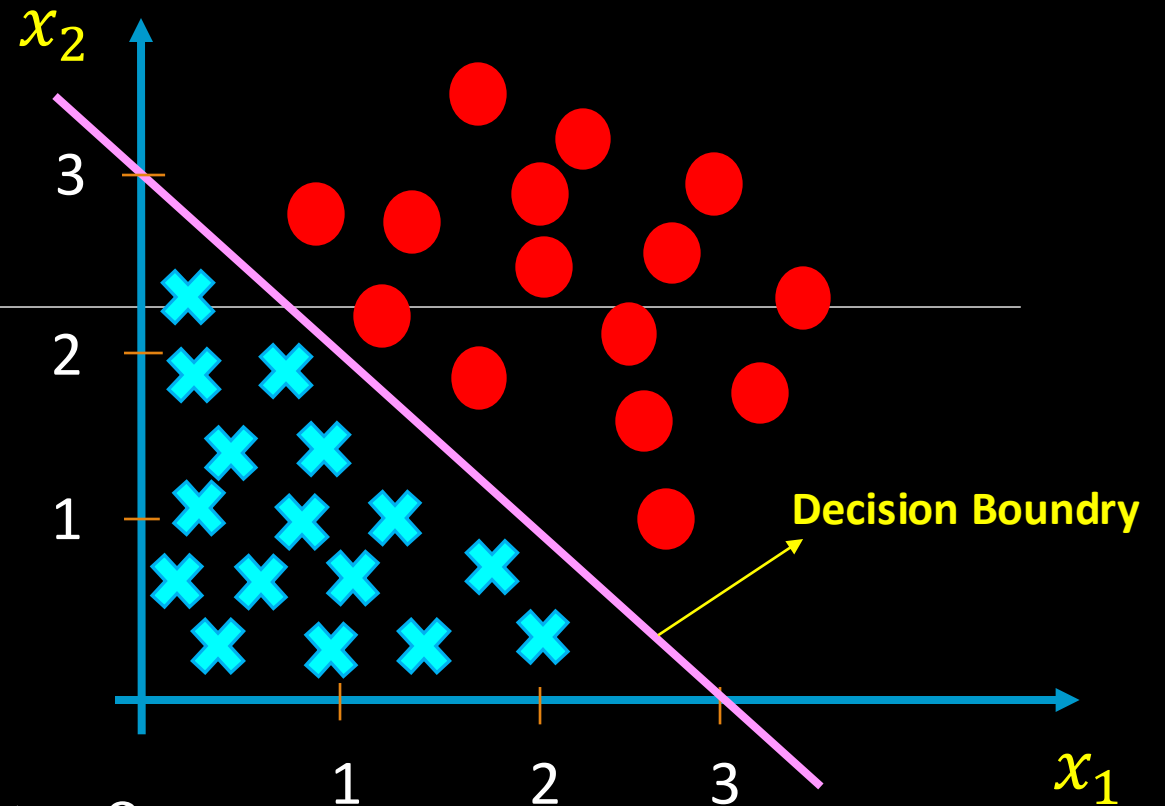
Prediction ' $y = 1$ ' If  $-3 + x_1 + x_2 \geq 0$



$$x_1 + x_2 \geq 3$$

' $y = 0$ '

$$x_1 + x_2 < 3$$



$$x_1 + x_2 = 3$$

$$h_{\theta}(x) = 0.5$$

# Cost Function

---

**Dataset:**  $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

$$x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix}$$

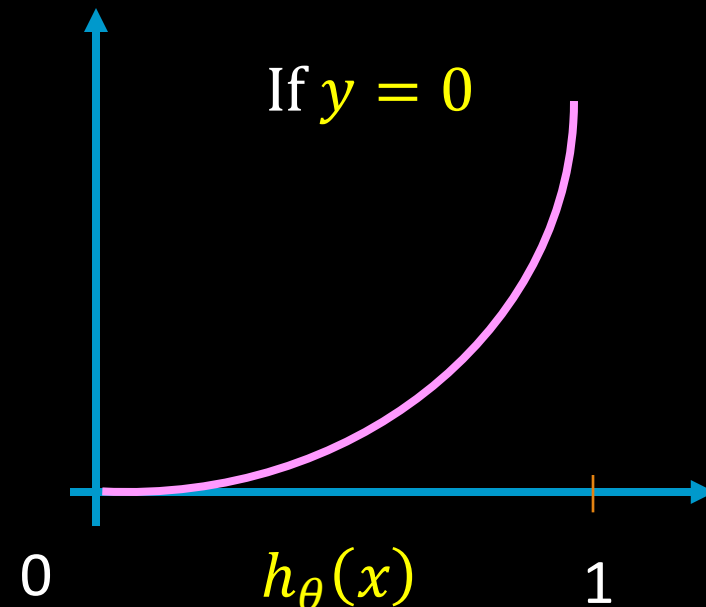
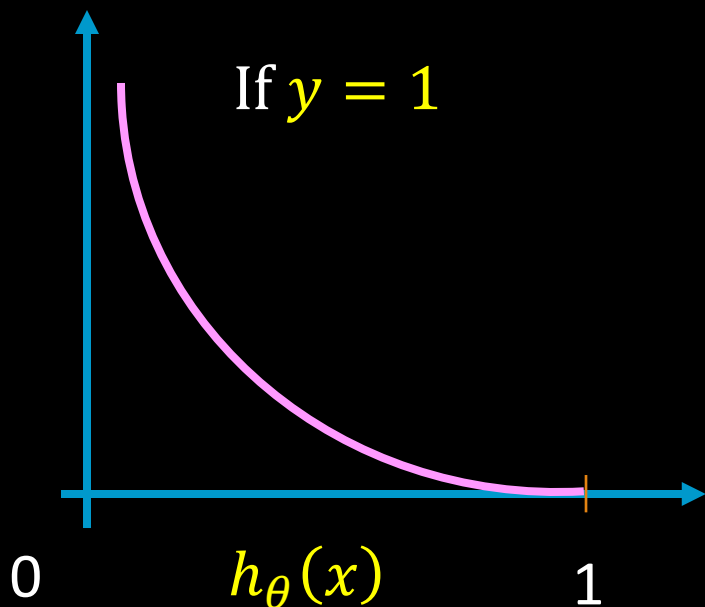
$$x_0 = 1, y \in \{0, 1\}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-x \cdot \theta}}$$

**How do we choose  $\theta$ ?**

# Cost Function

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



# Cost Function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

$$\text{If } y = 1, \quad \text{Cost}(h_{\theta}(x), y) = -\log(h_{\theta}(x))$$

$$\text{If } y = 0, \quad \text{Cost}(h_{\theta}(x), y) = -\log(1 - h_{\theta}(x))$$



# Gradient Descent

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$$

**Goal** :  $\min_{\theta} J(\theta)$ :

**Repeat** {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}



$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

# Logistic Regression

---

(Dataset)

| $X$   |       |       | $y$ |
|-------|-------|-------|-----|
| $x_0$ | $x_1$ | $x_2$ | $y$ |
| 1     | 2     | 3     | 1   |
| 1     | 3     | 2     | 0   |
| 1     | 3     | 1     | 0   |
| 1     | 1     | 3     | 1   |
| 1     | 0     | 4     | 1   |

# Logistic Regression

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix} \quad y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad \alpha = 0.1$$

## Hypothesis

$$Z = X \cdot \theta$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-Z}}$$

# Logistic Regression

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\alpha = 0.1$$

**Hypothesis**

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$

$$Z = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\alpha = 0.1$$

$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Cost Function

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\alpha = 0.1$$

$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Cost Function

$$J(\theta) = -\frac{1}{5} \sum_{i=1}^5 \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \log \left( \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} \right) + \left( 1 - \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \right) \log \left( 1 - \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} \right)$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\alpha = 0.1$$

$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Cost Function

$$J(\theta) = -\frac{1}{5} \sum_{i=1}^5 \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} -0.693 \\ -0.693 \\ -0.693 \\ -0.693 \\ -0.693 \end{bmatrix} + \left( \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \right) \cdot \left( \begin{bmatrix} -0.693 \\ -0.693 \\ -0.693 \\ -0.693 \\ -0.693 \end{bmatrix} \right)$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad \alpha = 0.1$$

$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Cost Function

$$J(\theta) = -\frac{1}{5} \sum_{i=1}^5 \begin{bmatrix} -0.693 \\ 0 \\ 0 \\ -0.693 \\ -0.693 \end{bmatrix} + \left( \begin{bmatrix} 0 \\ -0.693 \\ -0.693 \\ 0 \\ 0 \end{bmatrix} \right) = -\frac{1}{5} \sum_{i=1}^5 \begin{bmatrix} -0.693 \\ -0.693 \\ -0.693 \\ -0.693 \\ -0.693 \end{bmatrix} = 0.693$$



# Logistic Regression

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\alpha = 0.1$$

**Hypothesis**

$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

**Cost Function**

$$J(\theta) = 0.693$$

## Gradient Descent

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Gradient Descent

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \cdot X^T \cdot (h(x) - y)$$

## Cost Function

$$J(\theta) = 0.693$$

$$\alpha = 0.1$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix}$$

## Cost Function

$$\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$
$$J(\theta) = 0.693$$
$$\alpha = 0.1 \quad h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Gradient Descent

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \cdot X^T \cdot (h(x) - y)$$

$$\theta_j = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \alpha \cdot \frac{1}{5} \cdot \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix}^T \cdot \left( \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \right)$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix}$$

## Cost Function

$$J(\theta) = 0.693$$
$$\alpha = 0.1$$
$$\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$
$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Gradient Descent

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \cdot X^T \cdot (h(x) - y)$$

$$\theta_j = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \alpha \cdot \frac{1}{5} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 2 & 3 & 3 & 1 & 0 \\ 3 & 2 & 1 & 3 & 4 \end{bmatrix} \cdot \begin{pmatrix} \begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ -0.5 \\ -0.5 \end{bmatrix} \end{pmatrix}$$

# Logistic Regression

## Hypothesis

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix}$$

## Cost Function

$$J(\theta) = 0.693$$
$$\alpha = 0.1$$
$$\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$
$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

## Gradient Descent

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \cdot X^T \cdot (h(x) - y)$$

$$\theta_j = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \alpha \cdot \frac{1}{5} \cdot \begin{bmatrix} -0.5 \\ 1.5 \\ -3.5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} -0.01 \\ 0.03 \\ -0.07 \end{bmatrix} = \begin{bmatrix} 0.01 \\ -0.03 \\ 0.07 \end{bmatrix}$$

# Logistic Regression

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \\ 1 & 0 & 4 \end{bmatrix}$$

$$\alpha = 0.1$$

## Iterations

### Iteration 1

$$\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$h_{\theta}(x) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

$$J(\theta) = 0.693$$

### Iteration 2

$$\theta = \begin{bmatrix} 0.01 \\ -0.03 \\ 0.07 \end{bmatrix}$$

$$h_{\theta}(x) = \begin{bmatrix} 0.54 \\ 0.515 \\ 0.498 \\ 0.547 \\ 0.572 \end{bmatrix}$$

$$J(\theta) = 0.638$$

### Iteration 3

$$\theta = \begin{bmatrix} 0.017 \\ -0.063 \\ 0.128 \end{bmatrix}$$

$$h_{\theta}(x) = \begin{bmatrix} 0.568 \\ 0.521 \\ 0.489 \\ 0.584 \\ 0.63 \end{bmatrix}$$

$$J(\theta) = 0.595$$

# Logistic Regression

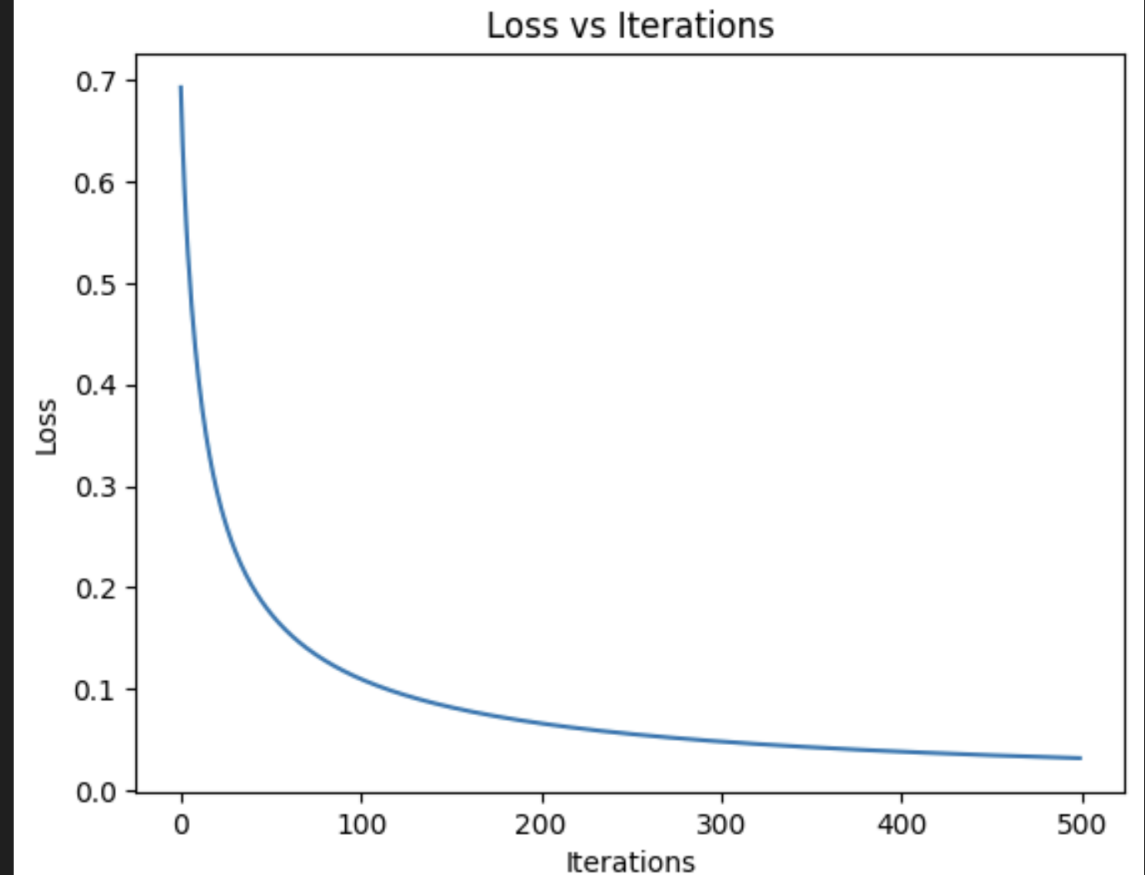
```
def sigmoid(z):  
    return 1/(1 + np.exp(-z))
```

✓ 0.0s

```
alpha = 0.1  
iterations = 500  
Loss = []  
  
m, n = X.shape  
  
theta = np.zeros((n, 1))  
  
for iter in range(iterations):  
    Z = np.dot(X, theta)  
    h_x = sigmoid(Z)  
    error = h_x - y  
    loss = -np.sum((y * np.log(h_x) + ((1-y) * np.log(1 - h_x))))/m  
    Loss.append(loss)  
    theta = theta - (alpha * np.dot(X.T, error))/m  
    print(f"{iter} iteration: Loss -> {loss}")
```

```
# Plot the loss over iterations  
plt.plot(range(iterations), Loss)  
plt.title("Loss vs Iterations")  
plt.xlabel("Iterations")  
plt.ylabel("Loss")  
plt.show()
```

✓ 0.0s



# Content

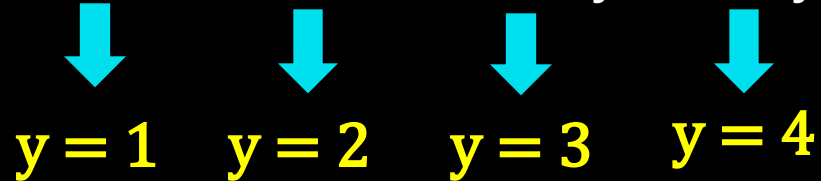
---

- ~~Logistic Regression~~
- Multi-class classification
- Regularization
- Implementation of Logistic Regression

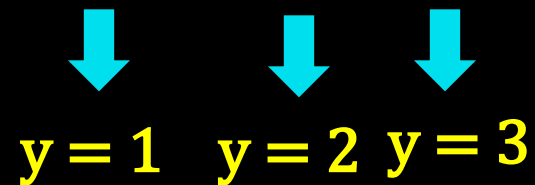


# Multi-class classification

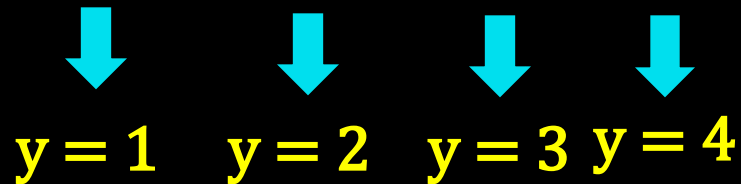
Email foldering/tagging: Work, Friends, Family, Hobby



Medical diagrams: Not ill, Cold, Flu

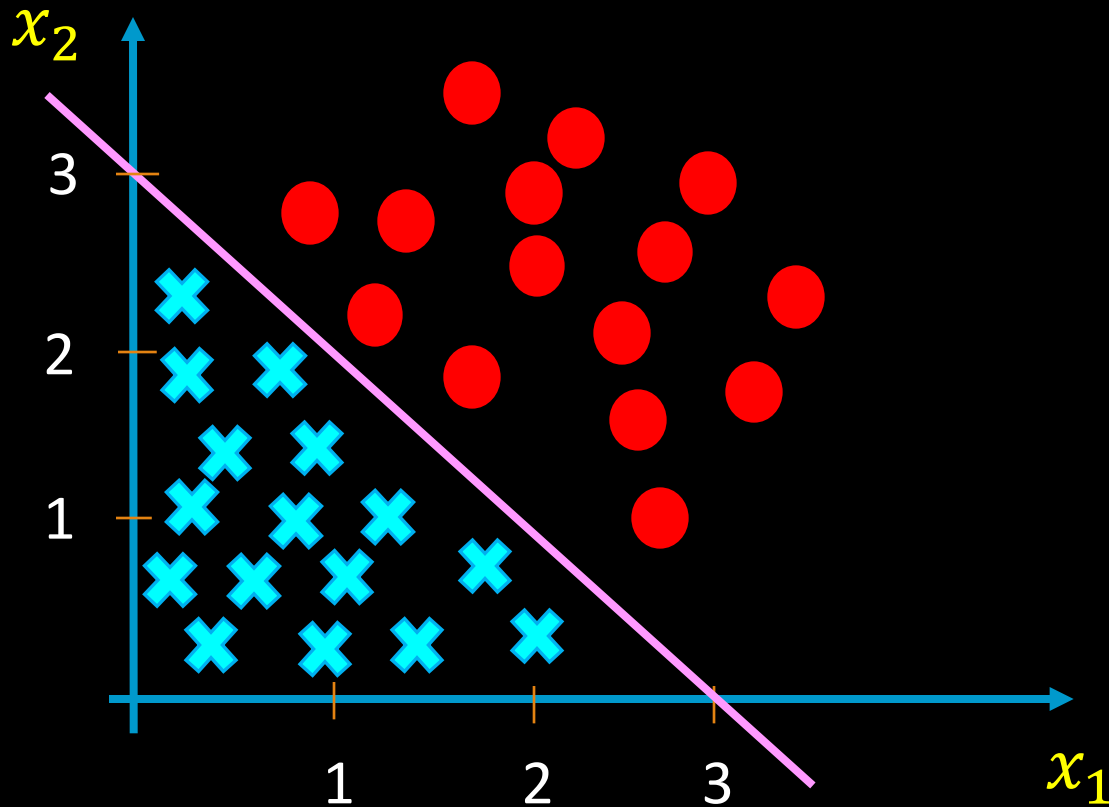


Weather: Sunny, Cloudy, Rain, Snow

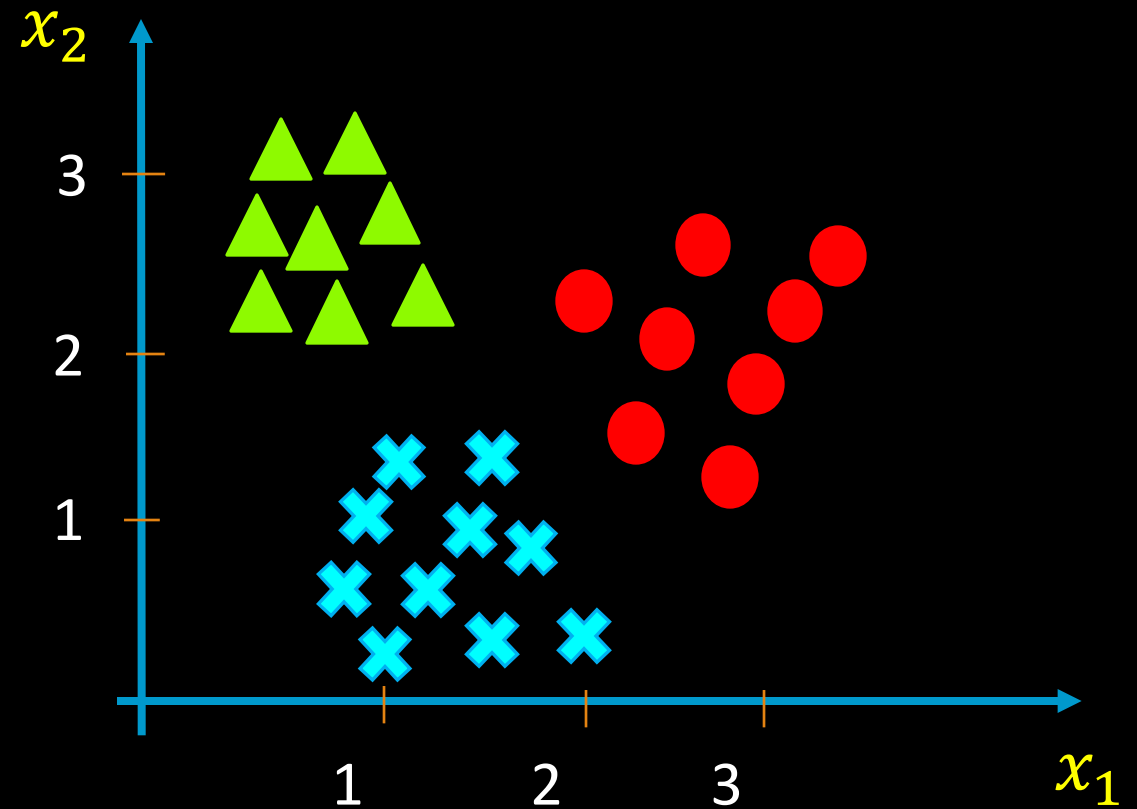


# Multi-class classification

## Binary Classification

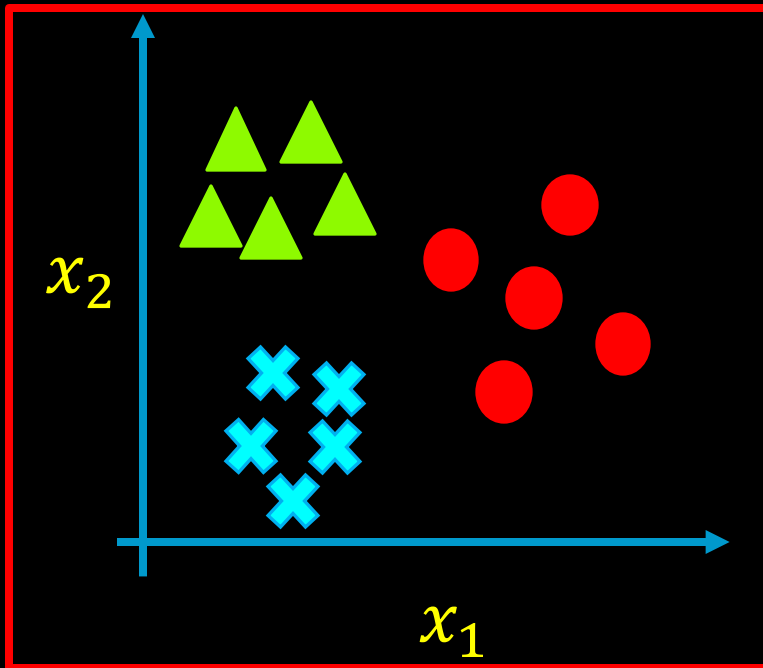


## Multi-class Classification



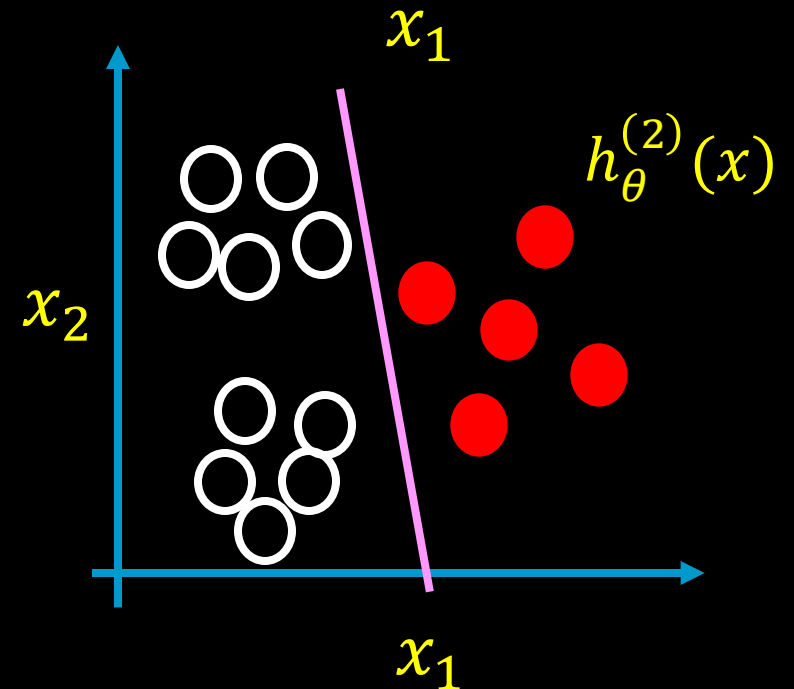
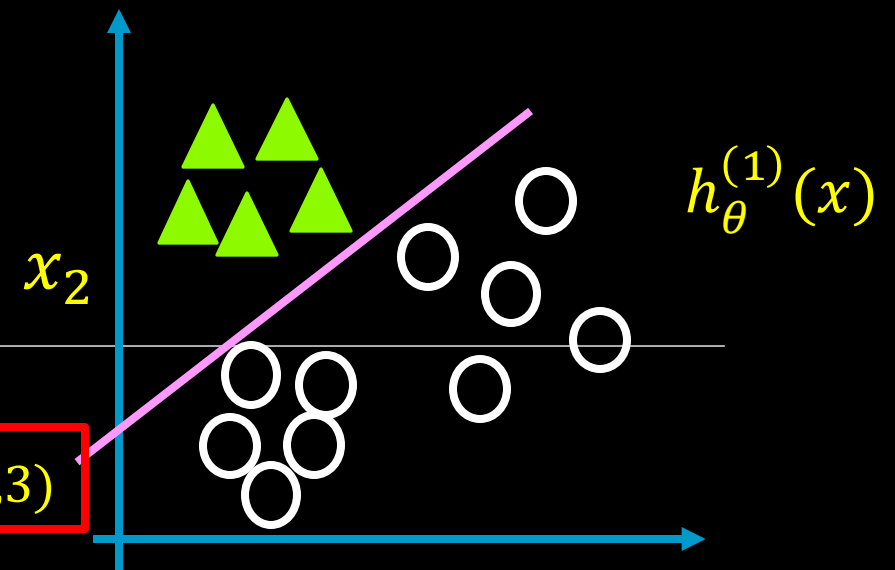
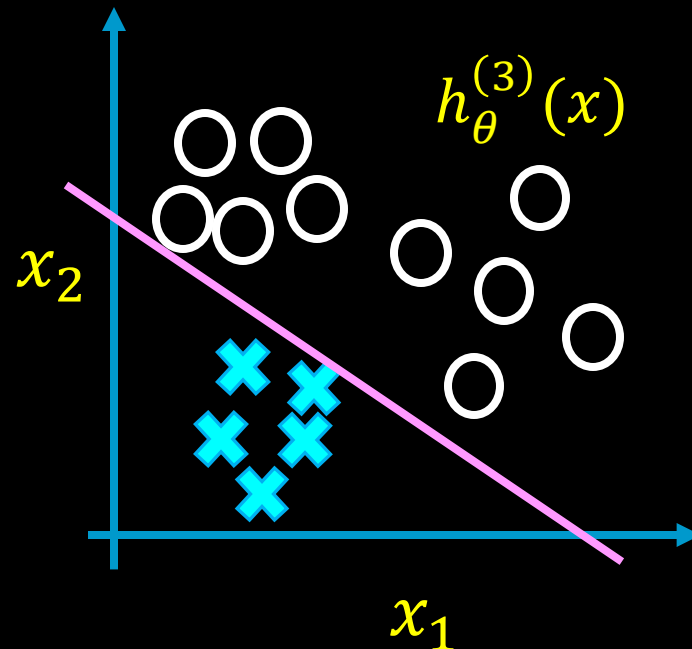
# Multi-class classification

## Multi-class Classification



- Class 1: ▲  
Class 2: ●  
Class 3: ×

$$h_{\theta}^{(i)}(x) = P(y = i|x; \theta) \quad (i = 1, 2, 3)$$



# Multi-class classification

---

Train a logistic regression classifier  $h_{\theta}^{(i)}(x)$  for each class  $i$  to predict the probability that  $y = i$ .

On a new input  $x$ , to make a prediction, pick the class  $i$  that maximizes

$$\max h_{\theta}^{(i)}(x)$$

# Multi-class classification

## Multi-Class Classification

```
import numpy as np
import matplotlib.pyplot as plt

# Creating a synthetic dataset for multi-class classification (3 classes)
# Let's assume we have 3 classes and 2 features
X = np.array([[1, 2], [2, 3], [3, 3], [5, 5], [2, 1], [3, 5],
              [2, 5], [1, 4], [5, 2], [4, 4]])
y = np.array([0, 1, 2, 1, 0, 2, 2, 0, 1, 1]) # 3 classes: 0, 1, 2

# Adding intercept term (bias) to the dataset
X = np.hstack([np.ones((X.shape[0], 1)), X]) # Adding a column of 1s
print(X.shape)
print(y.shape)
```

✓ 0.0s

(10, 3)

(10,)

```
# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Loss function (Binary Cross-Entropy)
def loss_function(h, y):
    return -np.mean(y * np.log(h) + (1 - y) * np.log(1 - h))

# One-vs-all logistic regression model training
def train_one_vs_all(X, y, num_labels, alpha, iterations):
    m, n = X.shape
    theta = np.zeros((num_labels, n)) # Initialize parameters for each class
    all_losses = []

    for c in range(num_labels):
        theta_c = np.zeros((n, 1)) # Initialize weights for class `c`
        y_c = np.where(y == c, 1, 0).reshape(-1, 1) # Convert y to binary for class `c`
        losses = []

        for i in range(iterations):
            z = np.dot(X, theta_c)
            h = sigmoid(z)
            error = h - y_c
            loss = loss_function(h, y_c)
            losses.append(loss)
            if i%100 == 0:
                print(f"{i} iteration, Loss: {loss}")

            # Gradient descent
            theta_c -= alpha * (np.dot(X.T, error)) / m

        theta[c, :] = theta_c.T
        all_losses.append(losses)

    return theta, all_losses

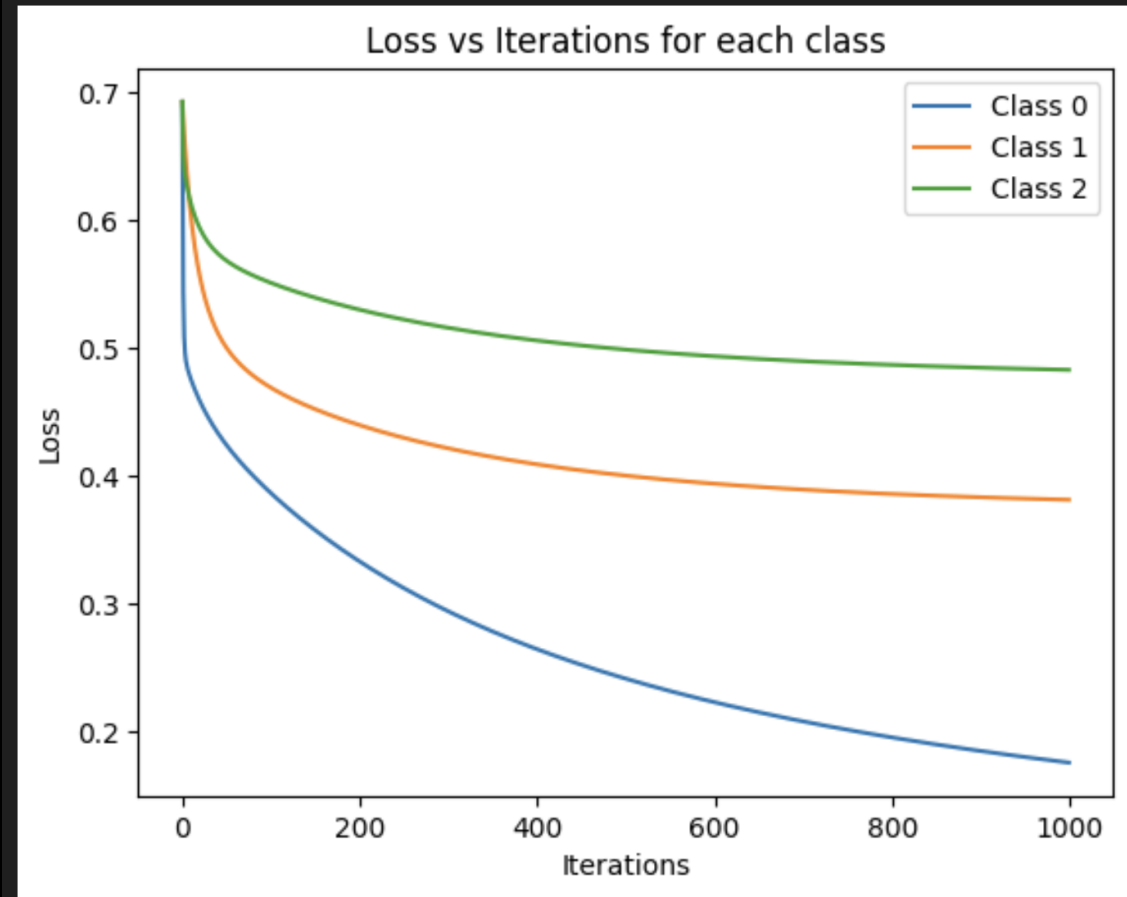
# Making predictions
def predict_one_vs_all(X, theta):
    z = np.dot(X, theta.T)
    h = sigmoid(z)
    return np.argmax(h, axis=1)
```

# Multi-class classification

```
# Training logistic regression models for each class
num_labels = 3 # We have 3 classes (0, 1, 2)
alpha = 0.1
iterations = 1000
theta, all_losses = train_one_vs_all(X, y, num_labels, alpha, iterations)
```

```
# Testing the model on new input
x_test = np.array([[1, 3, 3]]) # Add intercept term (bias)
pred = predict_one_vs_all(x_test, theta)
print(f"Predicted class for test sample: {pred[0]}")
```

```
# Plotting the loss for each class
for i in range(num_labels):
    plt.plot(range(iterations), all_losses[i], label=f'Class {i}')
plt.title("Loss vs Iterations for each class")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



# Content

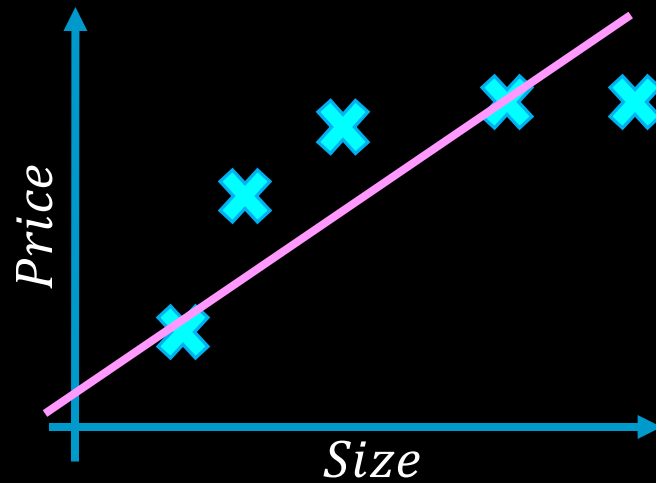
---

- ~~Logistic Regression~~
- ~~Multi-class classification~~
- Regularization
- Implementation of Logistic Regression

# Regularization

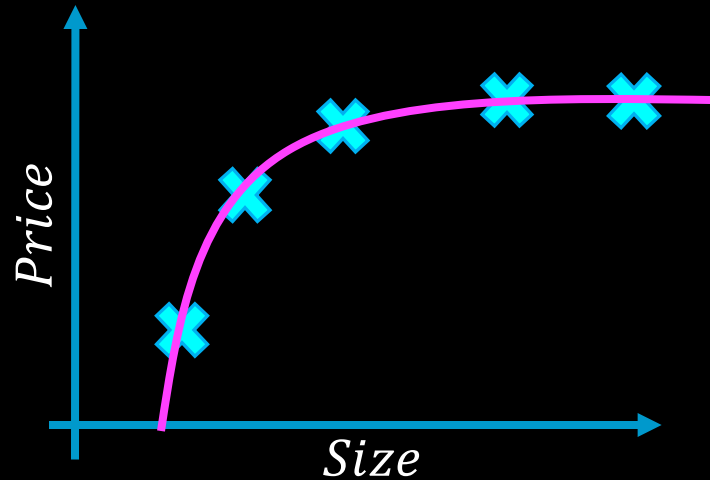
## The problem of overfitting

### Example: Linear Regression (housing prices)

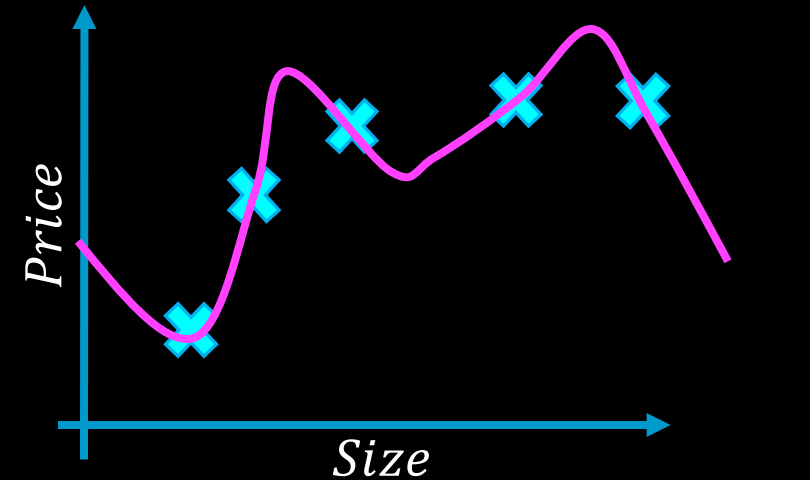


$$\theta_0 + \theta_1 x$$

Underfit -> High bias



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Overfitt -> High variance



# Regularization

## The problem of overfitting

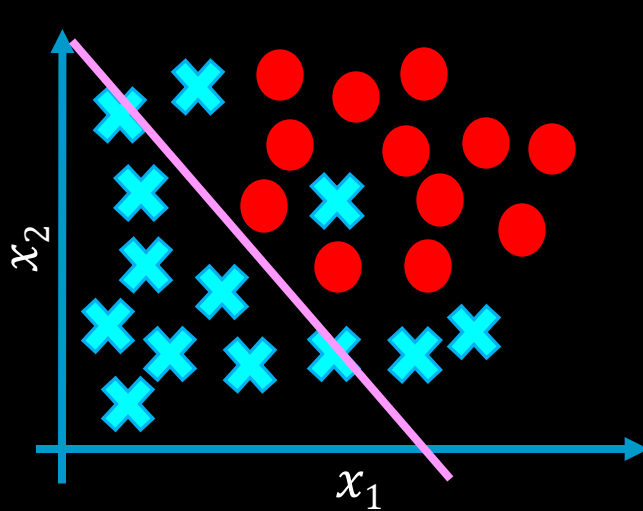
---

**Overfitting:** If we have too many features, the learned hypothesis may fit the training set very well ( $J(\theta) = \frac{1}{2m} \sum_{i=1}^m ((h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0)$ ) but fail to generalize to new examples (predict price on new examples).

# Regularization

## The problem of overfitting

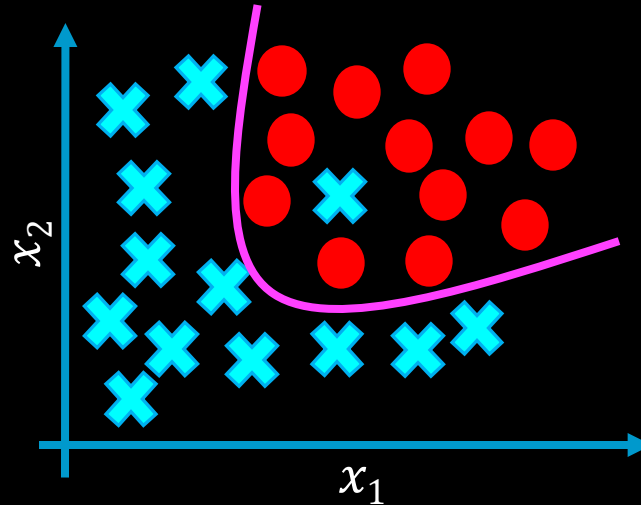
### Example: Logistic Regression



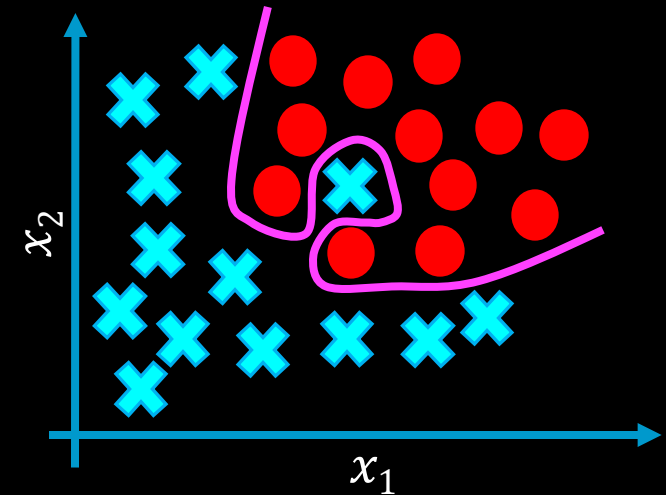
$$h_{\theta} = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$g$  = sigmoid function

**Underfit -> High bias**



$$h_{\theta} = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$



$$h_{\theta} = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

**Overfitt -> High variance**

# Regularization

## Addressing overfitting:

---

Options:

### 1. Reduce number of feature

- Manually select which features to keep.
- Model selection algorithm (later in course)

### 2. Regularization.

- Keep all the features, but reduce magnitude/values of parameters  $\theta_j$ .
- Works well when we have a lot of features, each of which contributes a bit to predicting  $y$ .

# Regularization

## Linear Regression

---

Cost function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2$$

Gradient Descent:

$$\theta_0 := \theta_0 - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \left( \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)} + \frac{\lambda}{m} \theta_j \right)$$

$\lambda$  = regularization parameter

# Regularization

## Logistic Regression

---

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Gradient Descent:

$$\theta_0 := \theta_0 - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \left( \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)} + \frac{\lambda}{m} \theta_j \right)$$

$\lambda$  = regularization parameter

# Content

---

- **Logistic Regression**
- **Multi-class classification**
- **Regularization**
- **Implementation of Logistic Regression**



